Módulo DS4 · Sistemas de recomendación

Ciencia de Datos e IA Aplicada: de los datos a las decisiones

N3-DS · Data Science avanzada · Popularidad · Filtrado colaborativo · Factorización de matrices · Evaluación · Arranque en frío

6.1 El problema de la recomendación y la matriz de valoraciones

¿Por qué recomendar?

- Catálogos de miles o millones de ítems
- Atención limitada: segundos en la pantalla de inicio
- Objetivo de negocio: más valor del catálogo → consumo, conversión, retención
- **Cola larga**: el 10 % de películas más vistas acumula el 38 % de las valoraciones (notebook)
- Un buen recomendador conecta a cada cliente con su nicho

La matriz de valoraciones R

	Película 1	Película 2	Película 3	Película 4
Ana	5	?	3	?
Bruno	?	4	?	?
Carmen	1	?	?	5

- Filas: usuarios (m) · Columnas: ítems (n) · Celdas: r_{ui}
- **Recomendar = rellenar los huecos y ordenar** lo no visto
- Huecos como `NaN`, nunca como 0

Una matriz casi vacía

Conjunto	Usuarios × ítems	Valoraciones	Densidad
Notebook	600 × 300	20.775	11,5 %
MovieLens 100K	943 × 1.682	100.000	6,3 %
E-commerce típico	millones × miles	—	< 1 %

- Pocos datos por usuario (28 de media en train) → **riesgo de sobreajuste**
- Pocos ítems en común para comparar usuarios

Los huecos no son aleatorios

- La gente ve lo que cree que le gustará...
- ...y lo que le mostramos
- En el notebook: $P(\text{ver}) \uparrow$ con popularidad y con la nota latente
- **Faltantes no al azar** (MNAR): la ausencia también informa
- Un modelo entrenado con lo observado hereda ese sesgo

Familias de recomendadores

Familia	Idea	Débil en
Popularidad	Lo más visto / mejor valorado	No personaliza
Contenido	Atributos parecidos a lo que te gustó	Poca sorpresa
Colaborativo	"A quien le gustó X, le gustó Y"	Arranque en frío
Híbrido	Combinación	Complejidad

Dos tareas distintas

Tarea	Pregunta	Métrica
Predicción	¿Qué nota daría u a i?	RMSE
Top-N	¿Qué N ítems le enseño?	precisión@k, NDCG

Un modelo puede acertar notas de películas mediocres y fallar en el top-5.

Preparar la evaluación (notebook, sección 2)

```
test = valoraciones.groupby("usuario", group_keys=False) \
                   .sample(frac=0.2, random_state=0)
train = valoraciones.drop(test.index)
R = np.full((n_usuarios, n_peliculas), np.nan)
R[train["usuario"], train["pelicula"]] = train["nota"]
```

- 20 % de las notas **de cada usuario** a test
- Todo (medias, sesgos, similitudes) se calcula con train

6.2 Referencias: popularidad y modelo de sesgos

¿Por qué empezar con referencias?

- Sin referencia no sabes si el modelo complejo aporta algo
- Dos referencias imprescindibles:
  - **Popularidad**
  - **Modelo de sesgos** $\mu + b_u + b_i$
- Regla: ningún modelo se despliega sin ganar a ambas

Popularidad

- Puntuación = nº de valoraciones (o ventas, reproducciones)
- Misma lista para todos
- ✓ Barata, estable, explicable, sirve el día 0
- ✘ No personaliza; refuerza lo ya popular

```
pred_popular = np.tile(np.bincount(train["pelicula"],
                                   minlength=n_peliculas), (n_usuarios, 1))
```

Media bruta vs. media encogida

Ordenar por media bruta → gana la película con **una** nota de 5★

$$\tilde{r}_i = \frac{\sum_u r_{ui} + \lambda\,\mu}{n_i + \lambda}$$

- Pocas notas → manda la media global μ
- Muchas notas → manda la media de la película
- λ = nº de "notas ficticias" de valor μ

El modelo de sesgos

$$\hat{r}_{ui} = \mu + b_u + b_i$$

- μ: media global (3,75)
- b_u: usuario generoso (+) o exigente (–) · rango –0,85 a +0,80
- b_i: película mejor (+) o peor (–) · rango –1,25 a +1,09
- Ejemplo: $3{,}75 - 0{,}4 + 0{,}8 = 4{,}15$

Estimación regularizada

$$b_i = \frac{\sum_{u \in U(i)} (r_{ui} - \mu - b_u)}{|U(i)| + \lambda} \qquad b_u = \frac{\sum_{i \in I(u)} (r_{ui} - \mu - b_i)}{|I(u)| + \lambda}$$

- Se alterna 10 veces (`sesgos(R, lam=5)`)
- El $+\lambda$ encoge a quien tiene pocas notas
- Película con 5 notas → sesgo a la mitad

Resultados (notebook, sección 6)

Modelo	RMSE	precisión@5
Media global	1,091	0,299 (≈ azar)
Popularidad	—	0,306
Sesgos	**0,957**	**0,361**

- –12 % de error sin personalizar
- MovieLens 100K (Surprise): sesgos 0,944 vs. SVD 0,934

Lecciones

- Los sesgos explican **la mayor parte de lo explicable**
- La popularidad ≈ azar *en esta métrica* (solo reordena lo ya visto)
- …pero es la base del arranque en frío (6.7)
- Errores típicos: no regularizar · referencias débiles · fuga de test

6.3 Filtrado colaborativo usuario-usuario e ítem-ítem

La intuición

- **Usuario-usuario**: pregunta a gente con gustos parecidos
- **Ítem-ítem**: mira películas parecidas a las que te gustaron
- "Parecido" = valorado de forma parecida por los mismos usuarios
- No necesita saber nada del contenido

Usuario-usuario

$$\hat{r}_{ui} = b_{ui} + \frac{\sum_{v \in N_k(u;i)} s_{uv} \left(r_{vi} - b_{vi}\right)}{\sum_{v \in N_k(u;i)} |s_{uv}|}$$

1. Similitud de u con los demás usuarios
2. k vecinos más parecidos que vieron i
3. Media ponderada de sus **desviaciones**

Ítem-ítem

$$\hat{r}_{ui} = b_{ui} + \frac{\sum_{j \in N_k(i;u)} s_{ij} \left(r_{uj} - b_{uj}\right)}{\sum_{j \in N_k(i;u)} |s_{ij}|}$$

- Vecinos = películas parecidas a i que u ya valoró
- "Quien compró esto también compró..."

Similitud del coseno sobre residuos

$$s_{ij} = \frac{\sum_u e_{ui}\, e_{uj}}{\sqrt{\sum_u e_{ui}^2}\sqrt{\sum_u e_{uj}^2}}, \qquad e_{ui} = r_{ui} - b_{ui}$$

- Restar sesgos ≈ correlación de Pearson
- Vectorizado: $E^\top E$ da todas las similitudes de golpe

Dos ajustes imprescindibles

- **Encogimiento por soporte**: $s_{ij} \cdot \dfrac{n_{ij}}{n_{ij} + \lambda}$
  - n_{ij} = usuarios en común · `encogimiento=10`
  - 10 usuarios en común → conserva la mitad
- **Solo k vecinos** (`k_vecinos=30`): evita diluir la señal

El código (notebook, sección 4)

```
residuos = np.where(mask, R - pred_sesgos, 0)
co = mask.T.astype(float) @ mask.astype(float)
normas = np.sqrt((residuos ** 2).sum(0))
sim = (residuos.T @ residuos) / (np.outer(normas, normas) + 1e-9)
sim *= co / (co + encogimiento)
np.fill_diagonal(sim, 0)
```

Resultados

Modelo	RMSE	precisión@5
Sesgos	0,957	0,361
Usuario-usuario (ej. 3)	0,856	0,388
Ítem-ítem	**0,813**	**0,390**

- Película: ~55 notas en train · Usuario: ~28
- Más datos por par → similitudes más fiables

Coste: 10 M usuarios, 50.000 productos

	Usuario-usuario	Ítem-ítem
Pares	10^{14}	$2{,}5 \cdot 10^{9}$
Matriz float32	400 TB	10 GB
Estabilidad	Cambia con cada clic	Precalculable cada noche
Explicación	"Gente como tú"	"Porque viste X"

En la práctica: solo los 50–100 vecinos por ítem + búsqueda aproximada.

Errores frecuentes

- Notas brutas en vez de residuos
- No encoger similitudes con poco soporte
- Recomendar lo ya consumido
- Usuario-usuario a gran escala sin necesidad

6.4 Factorización de matrices: SVD y ALS

Gustos latentes

```
   R (600 × 300)    ≈     P (600 × k)   ×   Qᵀ (k × 300)
usuarios × películas     gustos              características
```

- p_u: gustos del usuario · q_i: "retrato" de la película
- $k = 8$: 7.200 números en lugar de 180.000 celdas
- Los factores **no tienen nombre**

El modelo

$$\hat{r}_{ui} = \mu + b_u + b_i + p_u^\top q_i$$

- Sesgos: lo general
- $p_u^\top q_i$: el encaje personal
- Película similar = q_i cercano (coseno)

¿Y la SVD clásica?

- $R \approx U_k \Sigma_k V_k^\top$ (como la PCA del módulo 02)
- Necesita la matriz **completa** → aquí falta el 88 %
- Rellenar con ceros o medias distorsiona
- "SVD" en recomendación = ajustar **solo lo observado** (Premio Netflix)

Función objetivo

$$\min_{P,Q} \sum_{(u,i)\ \text{obs.}} \left(r_{ui} - \mu - b_u - b_i - p_u^\top q_i\right)^2 + \lambda \left(\sum_u \|p_u\|^2 + \sum_i \|q_i\|^2 \right)$$

- Sin λ: 8 parámetros por usuario con ~28 notas → memoriza ruido

ALS: mínimos cuadrados alternados

Con Q fijo, cada p_u es una regresión ridge:

$$p_u = \left(Q_{I(u)}^\top Q_{I(u)} + \lambda I\right)^{-1} Q_{I(u)}^\top \, e_{u,I(u)}$$

```
for u in range(R.shape[0]):
    idx = mask[u]
    P[u] = np.linalg.solve(Q[idx].T @ Q[idx] + I, Q[idx].T @ Rc[u, idx])
```

Luego P fijo → resolver Q. Repetir.

SGD frente a ALS

	SGD	ALS
Paso	Nota a nota	Usuario / ítem completo
Hiperparámetros	Tasa, épocas, λ	λ, iteraciones
Paralelizar	Delicado	Trivial
Datos implícitos	Costoso	Muy eficiente

Elegir k y λ

k	1	2	4	8	16	32
RMSE test	0,897	0,847	**0,743**	0,746	0,748	0,749

λ	0,1	1	3	8	30
RMSE train	0,357	0,366	0,438	0,606	0,900
RMSE test	0,869	0,687	**0,678**	0,746	0,950

Sobreajuste ← λ pequeño · λ grande → infraajuste

Comparación final (notebook, sección 6)

Modelo	RMSE	precisión@5
Media global	1,091	0,299
Sesgos	0,957	0,361
Ítem-ítem	0,813	0,390
ALS (k=8)	**0,746**	**0,399**

Usos de los factores

- **Recomendar**: ordenar $\hat{r}_{ui}$ de lo no visto (sección 8)
- **Similares**: vecinos en el espacio de q_i
- **Segmentar**: clustering de los p_u
- **Servir rápido**: producto escalar + búsqueda aproximada

6.5 Retroalimentación implícita (clics, visualizaciones)

Explícita frente a implícita

	Explícita	Implícita
Ejemplos	Estrellas, "me gusta"	Clics, minutos, compras
Volumen	Bajo	Muy alto
Negativos	Sí	Casi nunca
Ruido	Moderado	Alto

Preferencia y confianza

- No visto ≠ no le gusta (¿lo conoce? ¿está disponible?)
- **Preferencia**: $p_{ui} = 1$ si interactuó, 0 si no
- **Confianza**: $c_{ui} = 1 + \alpha\, r_{ui}$
  - r_{ui} = minutos, reproducciones, compras
- Hu, Koren y Volinsky (2008)

ALS ponderado

$$\min_{X,Y} \sum_{u,i} c_{ui} \left(p_{ui} - x_u^\top y_i\right)^2 + \lambda \left(\sum_u \|x_u\|^2 + \sum_i \|y_i\|^2 \right)$$

- Suma sobre **todas** las celdas
- Truco: $Y^\top Y$ se calcula una vez; solo se corrige con lo consumido
- Por eso ALS domina en datos implícitos a escala

En la práctica

```
import implicit
modelo = implicit.als.AlternatingLeastSquares(factors=64,
                                              regularization=0.05)
modelo.fit(matriz_usuario_item)     # dispersa, con confianzas
ids, punt = modelo.recommend(usuario, matriz_usuario_item[usuario], N=10)
```

También: Spark MLlib (`implicitPrefs=True`)

Otras ideas

- **Muestreo de negativos**: unas pocas no observadas por cada positiva
- **BPR**: aprender que lo consumido puntúa más que lo no consumido
- **Recencia**: pesa más lo reciente
- Evaluar con **ranking** (recall@k, NDCG) y partición **temporal**

La señal manda

Señal	Problema
Inició reproducción	*Autoplay*, portadas llamativas
Vio > 70 %	Más fiable
Minutos vistos	Premia lo largo
Abandono en 2 min	Señal negativa débil

Elegir la señal pesa más que elegir el algoritmo.

6.6 Evaluación offline y pruebas A/B de recomendadores

Tres niveles de evaluación

1. **Offline**: datos históricos · rápido · descarta ideas
2. **Online (A/B)**: usuarios reales · mide el efecto causal
3. **Estudios con usuarios**: confianza, sorpresa, satisfacción

Error frente a ranking

$$\mathrm{RMSE} = \sqrt{\frac{1}{|T|}\sum_{(u,i)\in T}\left(\hat{r}_{ui} - r_{ui}\right)^2}$$

- **Precisión@k**: relevantes entre los k primeros
- **Recall@k**: relevantes recuperados en el top-k
- **NDCG@k**: premia aciertos arriba

$$\mathrm{DCG@}k = \sum_{j=1}^{k} \frac{\mathrm{rel}_j}{\log_2(j+1)}$$

La precisión@5 del notebook

- Para cada usuario con ≥ 5 películas en test
- Ordena **esas** películas con el modelo, toma 5
- ¿Qué fracción tiene 5★?
- Limitación: solo reordena lo **ya visto**
- Por eso popularidad (0,306) ≈ azar (0,299)
- En producción: ordenar **todo** el catálogo no visto

Cómo partir los datos

Partición	Mide	Riesgo
Aleatoria por usuario	Completar huecos	"Ver el futuro"
Temporal	Despliegue real	Menos datos
Usuarios completos fuera	Arranque en frío	—

Fuga: sesgos, medias y similitudes **solo con train**.

Más allá de la precisión

- **Cobertura**: % del catálogo que llega a recomendarse
- **Diversidad**: cuán distintos son los ítems de una lista
- **Novedad**: cuán poco conocidos son
- **Serendipia**: aciertos que el usuario no habría encontrado

Prueba A/B de un recomendador

- Aleatorizar por **usuario**
- Métrica principal de negocio: minutos/usuario, conversión, retención 30 días
- Métricas de guarda: latencia, quejas, diversidad, coste de licencias
- Tamaño y duración **antes** de empezar (≥ 1 semana)
- Cuidado con el efecto novedad y el *peeking*
- Alternativa sensible: *interleaving*

Offline ≠ online

¿Mejor RMSE y mismas ventas?

- La mejora está en ítems que nunca llegan al top
- **Bucle de retroalimentación**: los datos los generó el sistema anterior
- Recomienda lo que el usuario habría encontrado igualmente

→ Decide con A/B, no con RMSE

6.7 Arranque en frío, diversidad y burbujas de filtro

Tres arranques en frío

Tipo	Falta	Estrategia
Usuario nuevo	b_u, p_u	Popularidad + contexto + *onboarding*
Ítem nuevo	b_i, q_i	Metadatos (híbrido) + exposición
Sistema nuevo	Todo	Contenido, reglas editoriales

Usuario nuevo (soluciones, ej. 1a)

Lista fija de 10	Cobertura	Acierto 5★	Nota media
Más populares	0,815	0,413	3,90
Mejor valoradas (b_i)	0,362	0,278	4,59
Media bruta	0,287	0,215	4,62
Híbrido	0,717	**0,418**	4,15

Después: p_u con Q fijo en cuanto haya unas pocas interacciones.

Ítem nuevo (soluciones, ej. 1b)

Metadatos → (b_i, q_i) con una regresión ridge

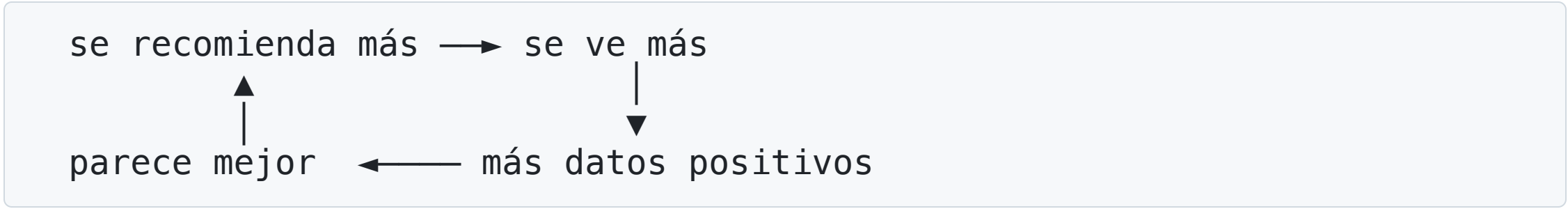
Predicción para 30 estrenos	**RMSE**
Media + b_u	1,076
+ q_i (géneros)	1,020
+ b_i (crítica)	0,923
Híbrido b_i + q_i	**0,855**

Géneros → *a quién* · Crítica → *cuánto*

Explorar frente a explotar

- Explotar: lo mejor según el modelo
- Explorar: aprender de lo incierto
- *ε-greedy*: con prob. ε, un candidato al azar
- UCB, muestreo de Thompson: exploran lo más incierto
- Ideal para estrenos

Sesgo de popularidad



- Baja la cobertura del catálogo
- Perjudica a proveedores pequeños

Diversificar la lista: MMR

$$\mathrm{MMR}(i) = \theta \cdot \mathrm{rel}(u, i) - (1 - \theta) \cdot \max_{j \in \mathrm{lista}} \mathrm{sim}(i, j)$$

- $\theta = 1$: ranking puro
- $\theta < 1$: penaliza lo repetitivo
- Otras: cuotas por categoría, límite por artista, huecos para novedades

Burbujas de filtro

- Solo ver lo que encaja con lo ya consumido → horizonte estrecho
- Entretenimiento: aburrimiento · Noticias: problema social
- Medidas: diversidad y cobertura en el panel, control del usuario, explicaciones
- UE (DSA): transparencia y opción no basada en perfiles para grandes plataformas

Lista de comprobación antes de desplegar

- ✔ Referencias (popularidad, sesgos) superadas
- ✔ Métricas de ranking + cobertura + diversidad
- ✔ Plan para usuario e ítem nuevos
- ✔ Exploración controlada
- ✔ Prueba A/B con métrica de negocio y de guarda
- ✔ Explicaciones y control para el usuario